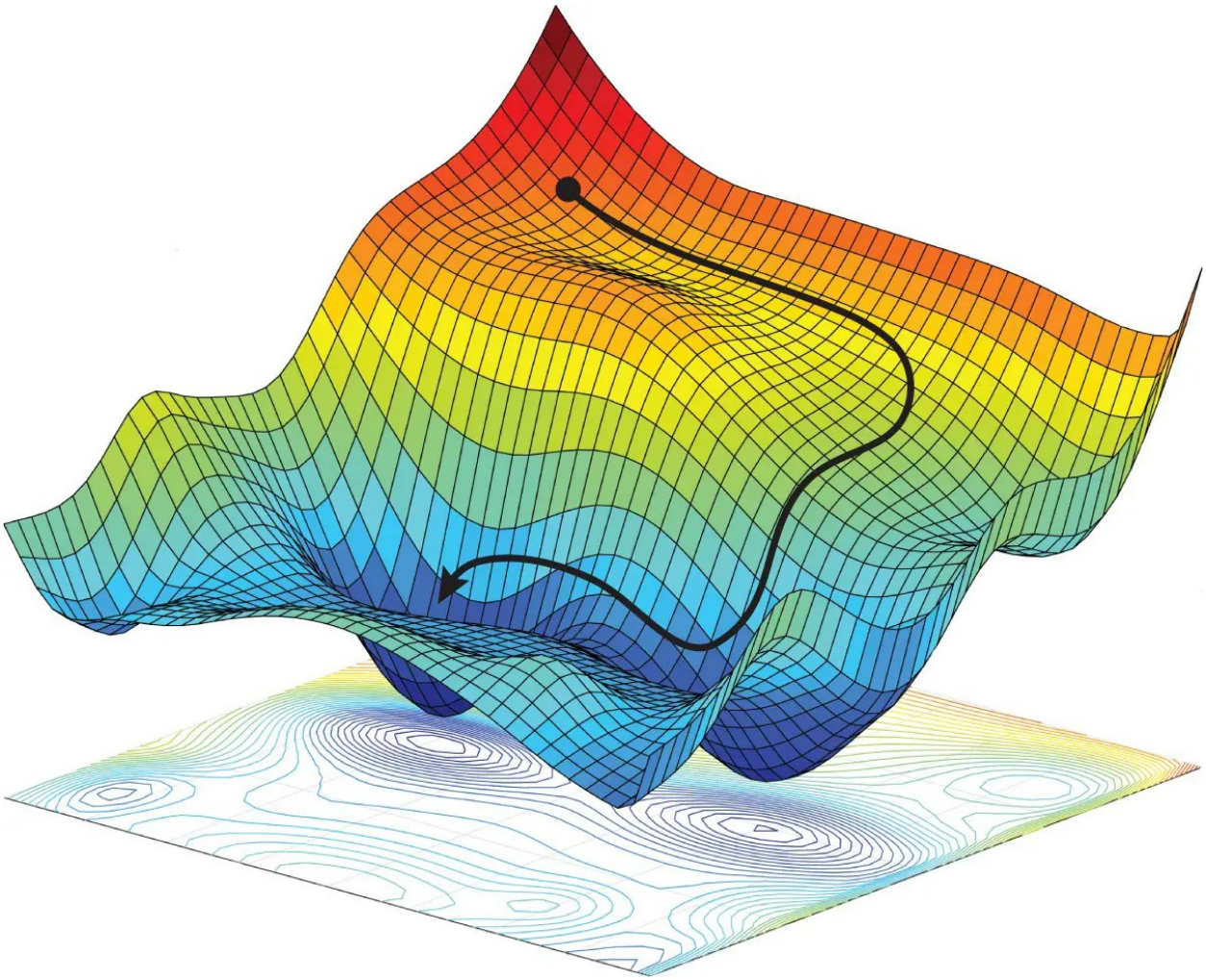


Local AI configuration

Alex Huhdanpaa

Purpose:

The purpose of this lab is to configure a local AI model to provide private Gen AI services to specific users on the local network of the AI workstation. This provides a whole host of benefits and can simplify operations, and is crucial to optimizing operations in an evolving technological environment

Definitions:

Gen AI:

Generative AI, like ChatGPT, creates new information through predictive guesses based on information provided in a prompt.

Tokens:

Sub-word bits and pieces of text and punctuation that are essentially the building blocks of language.

Embedding:

Assigning each token in a prompt a set of numbers, each corresponding to a different dimension, that are fine tuned to encode a meaning to the word

Vector:

A mathematical term that possesses both distance and direction, they can be broken down into distances in individual dimensions.

Compute:

Compute=time*processing power, it is a measure of how costly it is for a computer to complete a given task.

Open Source:

No restrictions on use or modification of a particular service, for example providing the source code for free to a specific program with no restrictions.

Matrix:

A two-dimensional set of numbers, think for example a table with a certain number of rows (one dimension) and a certain number of columns (the other dimension), that table is a matrix.

Tensor:

A tensor is similar to a matrix with one key difference, rather than only having 2 dimensions ever, a tensor can have any number of dimensions, for example think about data you collect of time of day, likability, location, and spontaneity. These could all be dimensions in a study for how likely a relationship is to start.

FlOp:

A Floating-Point Operation, it is any Add/Subtract/Multiply/Divide funtion involving numbers that have decimals.

LLM:

Large Language Model, a specific type of generative AI such as ChatGPT that offers predictive responses based on input queries.

Attention:

The cornerstone innovation of modern LLMs, it repeatedly uses the surrounding tokens to modify the embedding vector to refine the contextual understanding of a token.

Parameter:

A parameter is simply a decimal number between 1 and –1 that contains information based on training adjustments.

Quantization:

The process of reducing the size of an LLM by reducing the overall parameter count to fit on smaller hardware

MatMul:

A Matrix Multiplication is a type of operation in linear algebra that involves multiplying two different matrixes by each other, this is built up of multiplying and adding.

GPU:

Graphics Processing Unit, originally designed to calculate the many lines and geometric shapes in videogame rendering, GPUs have found new use in AI acceleration since they are able to compute simple calculations in a fast and efficient manner.

CPU:

Central Processing Unit, can be thought of as the brains of the computer, responsible for most tasks.

AAA:

Authentication, Authorization, Accounting. This describes the set of protocols used in networking to provide these services.

Frontend:

The client facing part of a program, for example a webpage

Backend:

The server facing part of a program, it handles databases, logic and other underlying infrastructures of a program.

RAG:

Retrieval-Augemented Generation, a system of making AI models more accurate by giving it a knowledge base pertaining to a specific industry or

role. This allows the AI to selectively take information from a new source while not having to go through a costly training stage.

Background Information:

AI Basics:

Generative Transformer AI models are what in the modern day most people consider “AI” this is because they are the most common consumer facing models that people see, US based examples of these include ChatGPT by OpenAI, Gemini by Google, Copilot (ChatGPT) by Microsoft, Grok by xAI and, importantly for us, Llama by Meta which we use for running on this lab due to the fact that it is open source, can run on cheaper hardware, and has wide support among the free to use/open source community while still being able to go toe-to-toe with the cutting edge models of its competitors.

At a surface level, all generative AI models do is predict text based on a prompt. These models can be understood as simple relationship programs that complete many Multiply/Add calculations per second to modify understanding. The core of this is tokens, which are often sub-word bits and pieces of text that have a high dimensional vector that encodes their semantic understanding; to improve understanding I will refer to tokens as words which is technically inaccurate. In the case of our model used in this lab, Llama 3.1:8B – an 8 billion parameter model, each of these words has a vector with 4096 dimensions and is able to somehow accurately encode the understanding of a specific word into the many dimensions of that vector.

Keeping it simple, AI models then compare each word to the words before it in an attempt to enrich the understanding of each word in context by changing the strength of the embedded value in each of the dimensions of the word’s vector. This allows the model to provide more accurate predictions of the next word. At a high level this is done by many multiplication and addition functions. For a standard generation of a single word in our quantized, lightweight model (8 out of 405 billion total parameters) there is about 17 trillion of these operations, as you might assume this means that the computational workload is very high, but interestingly it also ends up being almost entirely simple multiplication and

addition functions, meaning that dedicated hardware could likely allow for it to progress much faster.

In summary, we are running an AI chatbot, they work by completing repetitive arithmetic tasks on word like pieces of text, this large number of similar arithmetic tasks should be able to be done by dedicated hardware capable of high speed, highly parallelized solving.

GPUs:

Thankfully a solution to this problem was accidentally invented in 1999 when NVIDIA invented the GPU (Graphics Processing unit) which was designed to solve enormous numbers of FlOps (Floating Point Operations), such as multiplication or addition per second to compute graphics rendering, as vectors are needed to compute where light will travel and how it will be visible to the human eye. This also worked to solve the large number of MatMul (Matrix Multiplication) functions required by Neural Networks and later AI models as all these MatMul functions can be thought of as large packages of FlOps. This has led to an explosion in the complexity of these models in recent years as research evolved and caught up with hardware power and now means that ever more powerful GPUs were necessary to run these incredibly large models.

For example, the most powerful GPU currently is NVIDIA's Blackwell B200, which on its own can complete 20 quadrillion FlOps/s or in other words complete a standard generation of one word for our model in about a thousandth of a second. Thankfully for our wallets this is not needed in any corporate applications. For example, in this lab we use an RTX A4000 GPU which can complete about 153 Trillion FlOps/s or complete a standard generation of a word in roughly a tenth of a second if properly optimized. This means that it is at a rate well above what a human can generate and doesn't provide any meaningful friction to workflows while still retaining an acceptable price of ~$1100.

Given that Llama3.1:8B and other quantized models are still powerful it makes sense to equip employees with graphics cards, so they can utilize Gen AI to optimize their productivity by offloading tasks to the computer that otherwise may require significant time, but may not require keen insight, but as we'll see this is an imperfect solution that can be changed to offer better service at a lower cost.

To summarize, GPUs were originally invented to render graphics, but found a second use in AI roles, they can provide much better service at a lower cost than

other hardware options and equipping PCs with them means that employees are able to leverage AI locally.

Purpose of dedicated AI computers:

While it may be nice to have access to a personal GPU at all times, an important note is that for most of the day (if considered in terms of % of time) any given employee of a company doesn't need Generative AI services, so you end up spending a lot of money on putting a powerful GPU into every worker's computer that doesn't get used. This is obviously disadvantaging to any company that follows this route, as nobody should spend money where it is not needed.

To maximize return on investment one idea is to create a server that provides the service of Gen AI to employees while not being their local device, this means that for any given time there will be a predictable use of the services and you can use some simple math to find the number of GPUs it takes to meet this demand, you then can build an AI server on your company network to meet this demand and not leave unused processing power to sit idle. This way you can spend as little as possible while providing the best service to your employees.

You can, however, take this to another level. Say for example you have multiple departments in your company that have different needs: Human Resources needs an email copywriting AI to draft emails faster and with fewer errors to lighten the load and reduce the staffing costs in the department, while the IT department needs an AI assistant to help debug an issue on an OS (Operating System) deployment within the company. These two tasks require wildly different levels of model "Understanding". The email can be done by a model that emphasizes low compute and natural speech, while the debugging model must be done by a model with a strong understanding of operating system bugs, functionalities, and a keen attention to detail.

As you may be able to predict, the IT department will need more compute, and thanks to having a centralized powerful server you can switch between having a single powerful model or many weaker ones depending on network needs. This can allow for the flexibility needed for the variety of tasks required in modern businesses while saving money on further reducing the amount of hardware purchased.

To conclude, a dedicated local server confers many benefits over other deployments of local AI due to its increased flexibility, lower cost and ability to provide better service while rationalizing the high hardware costs.

Why not cloud based hardware:

Cloud computing can provide many advantages over conventional local computing, it can give you special hardware, such as very powerful computers when you need them without ever needing to buy or maintain any hardware, think of it like leasing a car but it is computer hardware. Furthermore, it can provide unparalleled flexibility if demand for a certain service is unreliable, such as if your marketing team does a great job and suddenly you need much more server space due to increased market demand. It can also reduce costs by decreasing your active server space during demand downturns. Cloud computing solves this by allowing for rapid expansion of server space through a number of ways. On top of all of this it provides more resilience against network or power outages and against natural disasters due to a global network of data centers. All of this begs the very logical question: Why would you not use cloud-based AI computing for your business?

There are a large number of advantages of local computing that the cloud cannot compete with. To start off, many industries, such as healthcare, defense, finance, and industries with IP-heavy trade secrets you cannot risk or are legally not allowed to move information past your local network, it cannot have internet access. Quite obviously the cloud does not permit these security needs.

Second, for latency sensitive applications, such as AI call customer support local machines are often the only solution since the local-cloud-local network latency is too high unless you pay for premium networking options which are often not fiscally reasonable. Additionally, latency can be increased if another customer of the same cloud services over-uses their resources and starts to back up the computing time you are paying for, another latency issue that is non-existent in local deployments.

Third, cost, in many situations the cost of local machines for AI applications specifically is cheaper. Mainly because the cost modelling of AI cloud infrastructure is designed for unpredictable or sporadic use where you may have long periods of downtime, rather than consistent demand that would warrant buying your own hardware to avoid long term costs. As a general rule if you are able to find a sweet spot where your hardware is being used more than 40% of the time you save money within a year.

Fourth, AI is a new industry, and the slow legislature has not yet caught up with regulation. AI companies are in constant need of high-quality data, and many queries provide just that, for example if you are working on a bug in proprietary code, the AI companies have all the legality in the world to take your code and use it to train their models, and thus end up distributing its ideas to every customer they have when they have when they make a new model. And your chat with the AI can be used as good quality reinforcement learning data, all this together means that using APIs provided by AI companies instead of any sort of cloud or local configurations means that you risk losing a valuable IP advantage over your competitors.

Additionally, it provides more flexibility and often cloud migration of one part of your tech stack means an integration if not migration for your entire stack which can mean customer service inconsistencies, high costs, and a lot of unnecessary work for marginal performance or cost improvements over time assuming you do find a niche in which you can operate cheaper.

As a final note however, oftentimes the most cost-efficient solution is to have a hybrid solution, where unreliable, sporadic, or experimental work is done on the cloud for upfront cost reduction and quick deployment, while reliable use is done on local hardware for long-term saving and the additional reasons above.

To wrap up, using a AI model locally often makes much more sense than running a server on the cloud. Namely among the reasons are increased control over security, conformance to regulation, cheaper in many cases, and increased flexibility. However, this does not discount cloud solutions, and a hybrid cloud infrastructure often makes the most sense for a company, of course, depending on the situation.

Networking Solutions:

Undeniably, centralizing your AI is going to cause some sort of networking difficulties, namely the fact that most solutions that provide flexibility also demand it. For example, in the dynamic model selection such as for the IT vs. HR department example provided above you are constantly shifting where to send your queries and can mean configuration of different customizable backends and use of libraries that you are unfamiliar with, all of this can mean that it is quite difficult to create these custom solutions as opposed to just using an API.

Second, if you take the logical decision of making multiple AI machines to promote reliability and lower initial response latency due to model loading, you can run into the difficulty of load balancing, and due to the discrete nature of GPU processing power it can mean that you end up with unused GPU power due to poor load balancing, this means that you must configure sophisticated load balancer rules to ensure that you maintain high availability at a low cost.

Third, while not strictly networking you must configure some form of AAA user authentication and access management to manage access to AI models that may contain proprietary information or for other reasons should not be accessed by the web.

In conclusion, running a local AI server can create some networking difficulties, but thankfully these issues are not new and generally there is good software support to easily smooth out any bumps in the deployment.

Docker:

Docker is a program developed by Docker Inc. Its primary functionality is to create, manage, run and distribute containers, which can be thought of as apps, they hold all the resources needed for a certain program to run and are used to create rapidly scalable, flexible programs. I personally used Docker Desktop, which is designed to run on Linux but, quite plainly I am using Windows 11 for this lab, so how does this work?

Thankfully Windows comes with a clever solution, known as WSL 2, or Windows Subsystem for Linux, which allows you to easily run Linux virtual machines. I installed Ubuntu on this which was the operating system that Docker then ran on.

Ollama:

Ollama is a program that allows users to run AI models locally on their own machine, it is notable for its simplicity, how user friendly it is; the fact that it is open source; and that it still holds a lot of functionality. It is very useful as a backend to run AI models partly due to its limited GUI, and partly due to its wide base of support

Open WebUI:

Open WebUI is another open-source platform designed to support local AI machines. Its function is primarily as a frontend to create a UI for the user and enable configuration of a lot of core features such as RAG and client authorization.

Summary:

In this lab I configure a local AI chatbot using Llama3.1:8B on an Ollama backend with an Open WebUI frontend running in a WSL Ubuntu docker container on Windows 11 I also configure a basic user database to permit specific users to access the Gen AI resources.

Commands:

POWERSHELL:

```
>Ollama run llama3.1
>wsl --install
>wsl --set-default-version 2
```

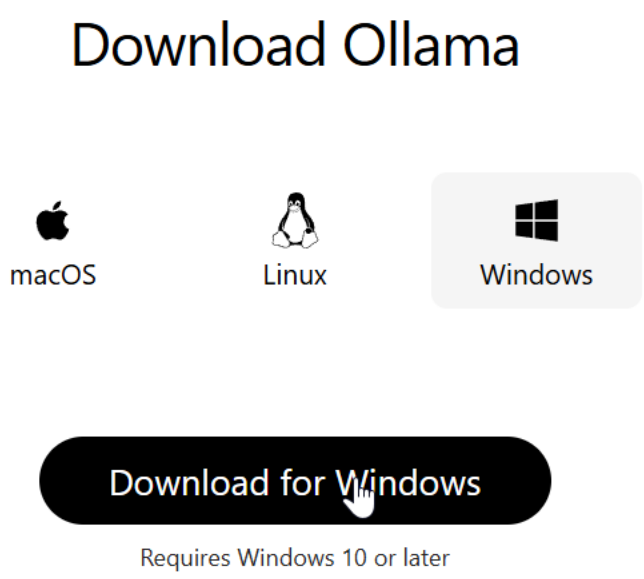
WSL UBUNTU:

```
>docker run -d -p 3000:8080 --gpus all --add-host=host.docker.internal:host-gateway -v open-webui:/app/backend/data --name open-webui --restart always ghcr.io/open-webui/open-webui:cuda
```

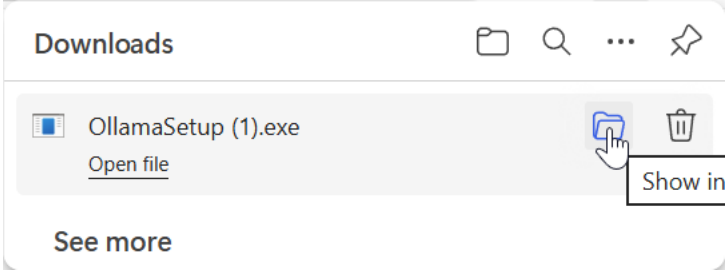
Screenshots:

OLLAMA:

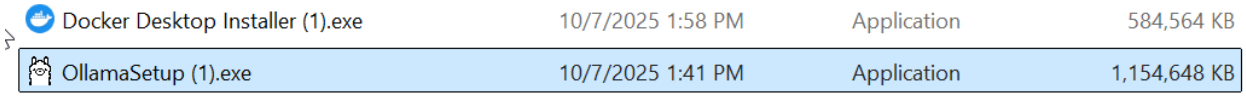
Go to https://ollama.com/download/windows
Click the download for windows button



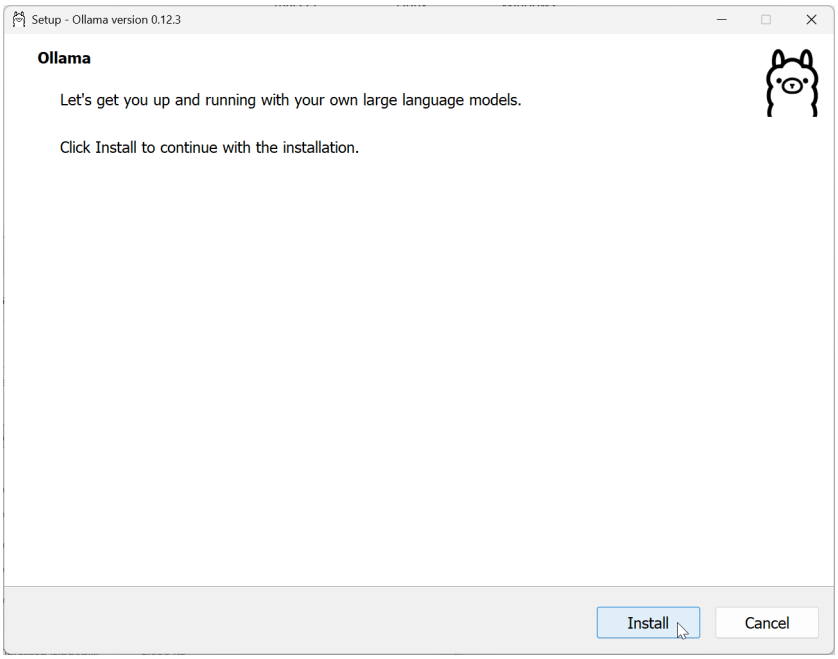
When the download finishes, open file location



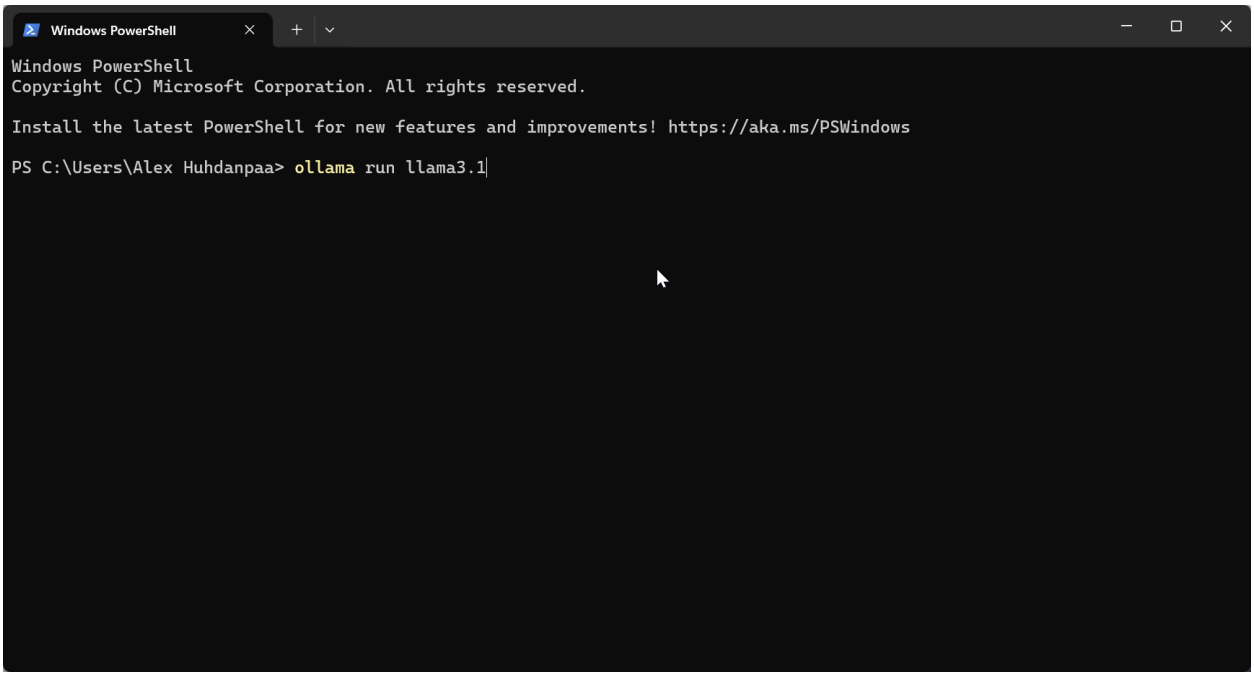
Double click on the file to open the installer



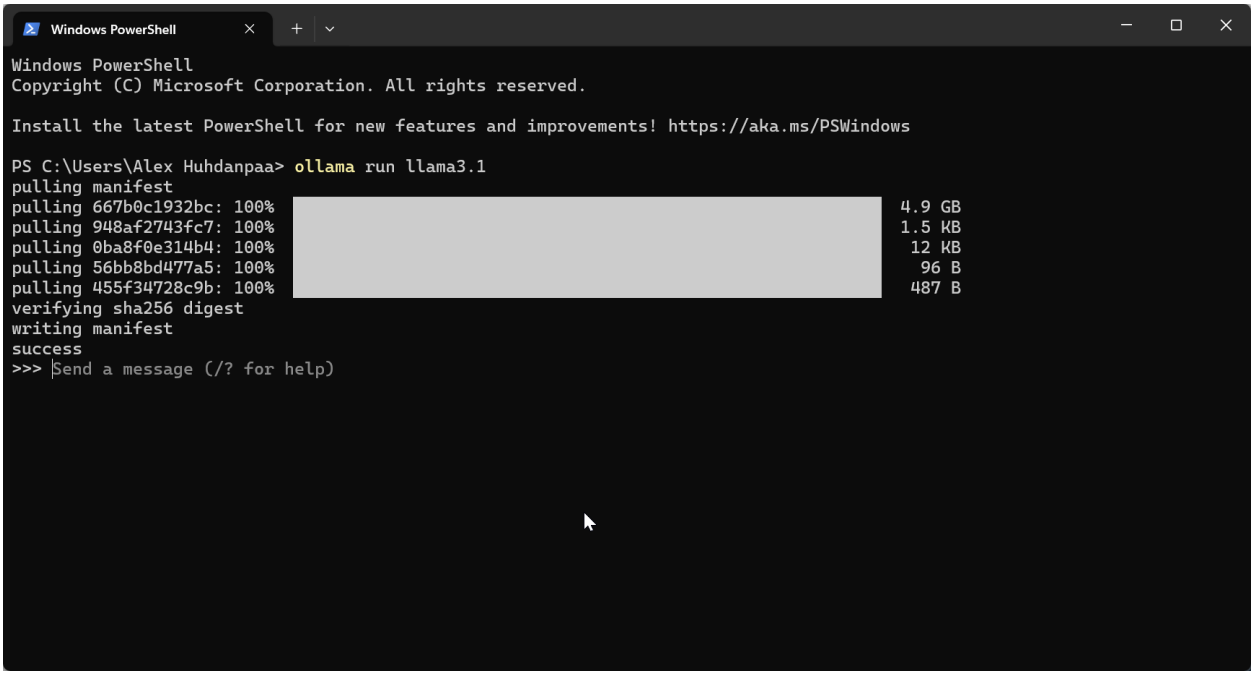
Click Install in the installer



Wait for the installer to finish and close, then open windows PowerShell, then enter the ollama run llama3.1 command



Wait for it to finish, and if successful you should have a text prompt like this



WSL:

Now install wsl with the wsl --install command

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Alex Huhdanpaa> wsl --install
Downloading: Ubuntu
[==================       32.8%                          ]
```

To ensure the latest version enter the wsl --set-default-version 2 command

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Alex Huhdanpaa> wsl --install
Downloading: Ubuntu
Installing: Ubuntu
Distribution successfully installed. It can be launched via 'wsl.exe -d Ubuntu'
PS C:\Users\Alex Huhdanpaa> wsl --set-default-version 2
```

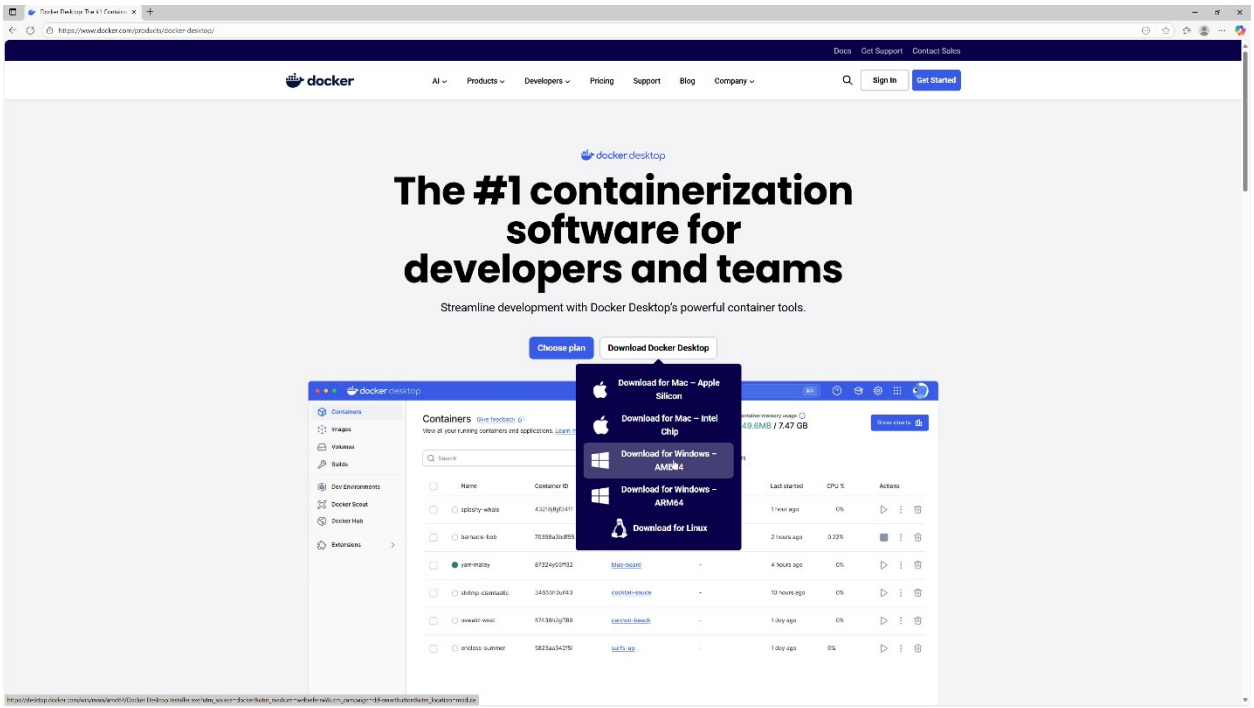
Create your new username and password

```
C:\Program Files\WSL\wsl.exe
Provisioning the new WSL instance Ubuntu
This might take a while...
Create a default Unix user account: wsluser
```

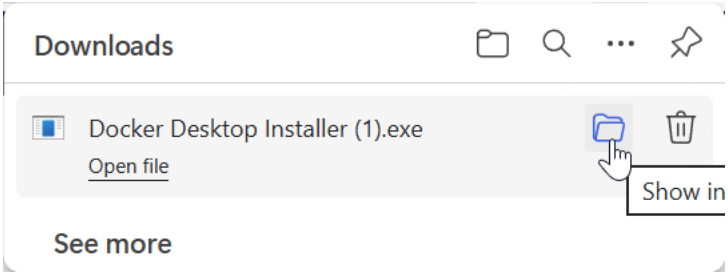
```
wsluser@DESKTOP-RDJTFSD:
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

wsluser@DESKTOP-RDJTFSD:~$ docker run -d -p 3000:8080 --gpus all --add-host=host.docker.internal:host-gateway -v open-we
bui:/app/backend/data --name open-webui --restart always ghcr.io/open-webui/open-webui:cuda
Unable to find image 'ghcr.io/open-webui/open-webui:cuda' locally
cuda: Pulling from open-webui/open-webui
d107e437f729: Downloading [==============================>                    ]  17.83MB/28.23MB
4f4fb700ef54: Download complete
659036234d55: Download complete
0c5c28ae5312: Download complete
1ce95d9ae66c: Downloading [=============================================>     ]  14.68MB/16.21MB
50a40c73fa38: Downloading [=================>                                 ]  13.63MB/39.63MB
35a59c4f5c0a: Download complete
2e1d87f9f991: Pulling fs layer
ce4392d98604: Download complete
84bf6814bdd8: Download complete
970a01e7ca52: Download complete
2fabda95f1fa: Downloading [>                                                  ]  1.049MB/91.1MB
60dbbb9770c7: Downloading [>                                                  ]  3.146MB/341.1MB
39c7879c2194: Download complete
89e26dfa6fb4: Download complete
```

Docker:

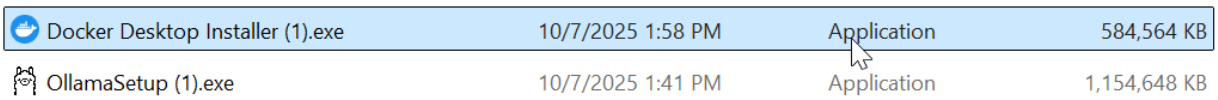
Go to https://www.docker.com/products/docker-desktop
Click on download docker desktop then on the version you need, for me this is Windows AMD64



Open File location



Double click on the installer to open it



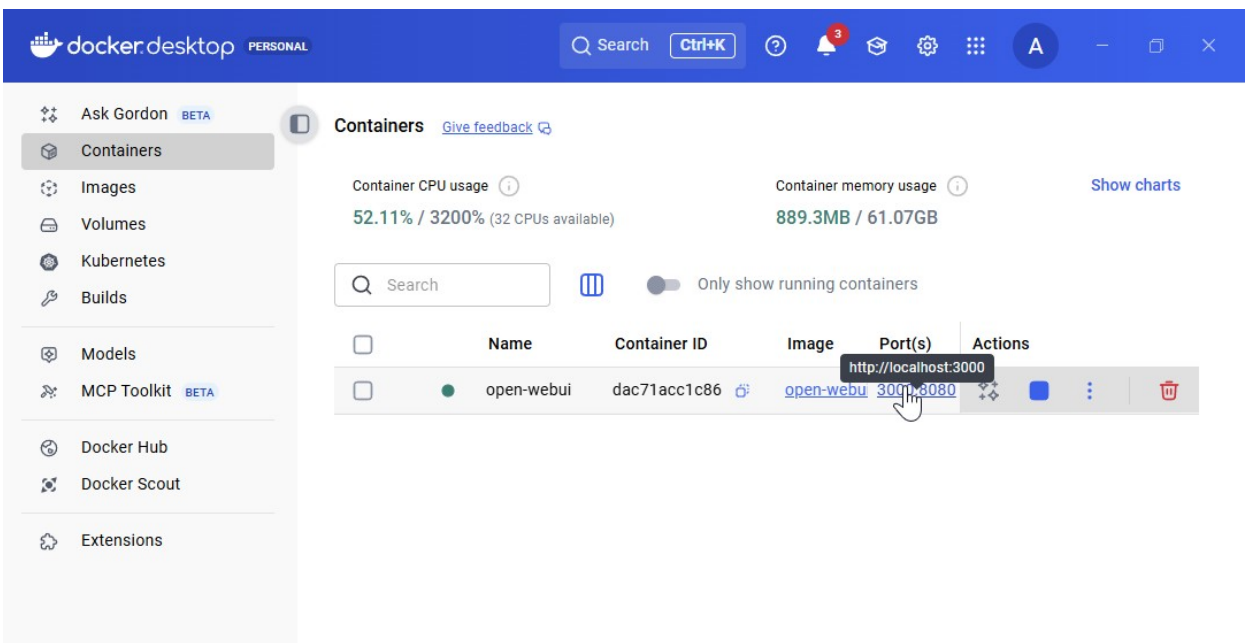
OPEN WEBUI:

Go to https://github.com/open-webui/open-webui

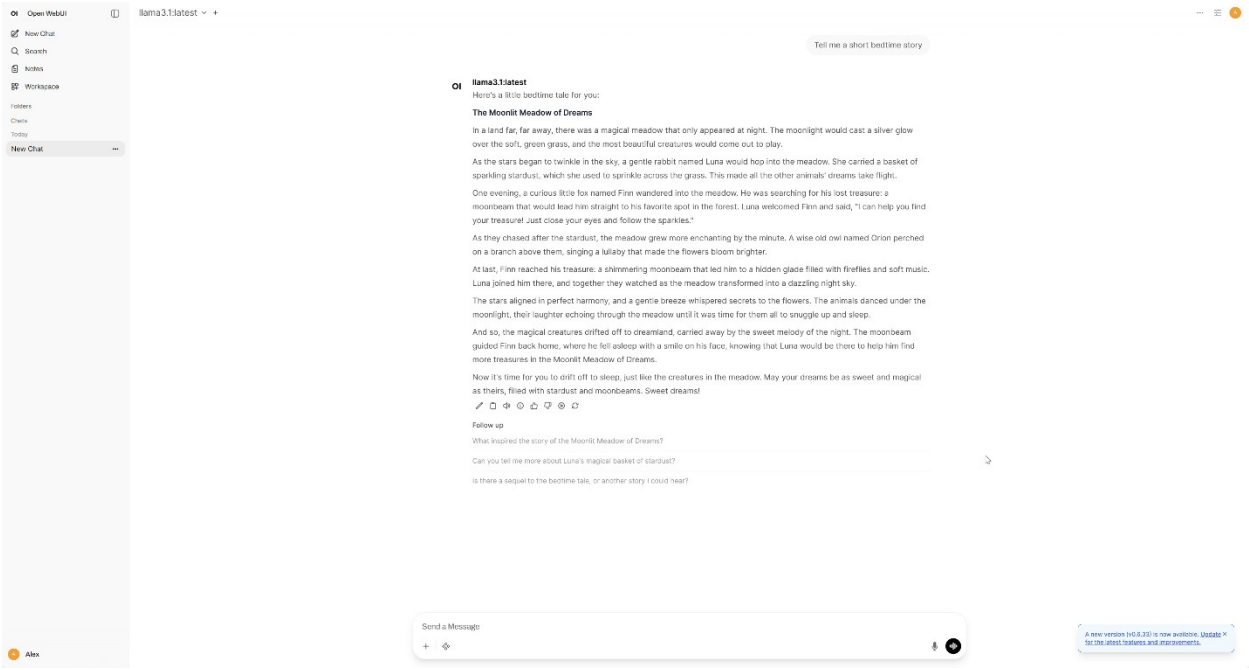
Once the installer has finished go to the WSL terminal again and enter the following command

```
wsluser@DESKTOP-RDJTFSD:
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

wsluser@DESKTOP-RDJTFSD:~$ docker run -d -p 3000:8080 --gpus all --add-host=host.docker.internal:host-gateway -v open-we
bui:/app/backend/data --name open-webui --restart always ghcr.io/open-webui/open-webui:cuda
Unable to find image 'ghcr.io/open-webui/open-webui:cuda' locally
cuda: Pulling from open-webui/open-webui
d107e437f729: Downloading [==============================>                    ]  17.83MB/28.23MB
4f4fb700ef54: Download complete
659036234d55: Download complete
0c5c28ae5312: Download complete
1ce95d9ae66c: Downloading [=============================================>     ]  14.68MB/16.21MB
50a40c73fa38: Downloading [=================>                                 ]  13.63MB/39.63MB
35a59c4f5c0a: Download complete
2e1d87f9f991: Pulling fs layer
ce4392d98604: Download complete
84bf6814bdd8: Download complete
970a01e7ca52: Download complete
2fabda95f1fa: Downloading [>                                                  ]  1.049MB/91.1MB
60dbbb9770c7: Downloading [>                                                  ]  3.146MB/341.1MB
39c7879c2194: Download complete
89e26dfa6fb4: Download complete
```

Once it has finished open Docker Desktop and navigate to containers and click on the 3000:8080 to open the local web page running the AI model



Now create an account and chat whatever you want to the AI model


Open WebUI
New Chat
Search
Notes
Workspace
Folders
Chats
Today
New Chat
Alex
llama3.1:latest
Tell me a short bedtime story
llama3.1:latest
Here's a little bedtime tale for you:
The Moonlit Meadow of Dreams
In a land far, far away, there was a magical meadow that only appeared at night. The moonlight would cast a silver glow over the soft, green grass, and the most beautiful creatures would come out to play.
As the stars began to twinkle in the sky, a gentle rabbit named Luna would hop into the meadow. She carried a basket of sparkling stardust, which she used to sprinkle across the grass. This made all the other animals' dreams take flight.
One evening, a curious little fox named Finn wandered into the meadow. He was searching for his lost treasure: a moonbeam that would lead him straight to his favorite spot in the forest. Luna welcomed Finn and said, "I can help you find your treasure! Just close your eyes and follow the sparkles."
As they chased after the stardust, the meadow grew more enchanting by the minute. A wise old owl named Orion perched on a branch above them, singing a lullaby that made the flowers bloom brighter.
At last, Finn reached his treasure: a shimmering moonbeam that led him to a hidden glade filled with fireflies and soft music. Luna joined him there, and together they watched as the meadow transformed into a dazzling night sky.
The stars aligned in perfect harmony, and a gentle breeze whispered secrets to the flowers. The animals danced under the moonlight, their laughter echoing through the meadow until it was time for them all to snuggle up and sleep.
And so, the magical creatures drifted off to dreamland, carried away by the sweet melody of the night. The moonbeam guided Finn back home, where he fell asleep with a smile on his face, knowing that Luna would be there to help him find more treasures in the Moonlit Meadow of Dreams.
Now it's time for you to drift off to sleep, just like the creatures in the meadow. May your dreams be as sweet and magical as theirs, filled with stardust and moonbeams. Sweet dreams!
Follow up
What inspired the story of the Moonlit Meadow of Dreams?
Can you tell me more about Luna's magical basket of stardust?
Is there a sequel to the bedtime tale, or another story I could hear?
Send a Message
A new version (v0.3.31) is now available. Update for the latest features and improvements.

Problems: